ALGORITHMS · PRINTABLE WORKBOOK

Proving, Counting, Searching

The skills a challenge set asks for, practised on problems of the same kind: writing an algorithm answer, "does it always exist?" with preferences that tie, counting loops that hide loops, counting proposals, and searching when you can only afford a few failures. 32 questions in five parts, with a key computed by running what each question describes.

How to use this packet

1. Work in order: each part is a Learn page, a warm-up, then questions.
2. Questions are the same *kind* as a challenge set's, never its own problems. The Learn pages stop short on purpose where the set would start.
3. Questions marked "open" want a written answer. The key shows one good answer; yours can differ and still be right.
4. Finish with the ready-check on the last page before the key.

Part 1	Writing an algorithm answer	5 questions
Part 2	Always, or not always?	7 questions
Part 3	Counting when the inner work is not constant	8 questions
Part 4	Counting proposals	5 questions
Part 5	Searching when failures are limited	7 questions
Summary	One page to keep	
Ready-check	What you should be able to do cold	
Key	Warm-ups and every question	

PART 1 · LEARN

Writing an algorithm answer

A design question is graded on four things, and a correct idea with a missing part loses the part. Write them as four labelled paragraphs.

Part	What it is	Test: could a classmate…
(a) Algorithm	The steps, in pseudocode or precise sentences.	…carry it out with no questions?
(b) Intuition	Why the idea should work, in a few sentences.	…say why you thought of it?
(c) Correctness	A proof about every input: an invariant, induction, or contradiction.	…check it line by line and be convinced?
(d) Running time	A bound, and why: count loops, cost per step.	…recount it and get the same bound?

Worked example: is a graph bipartite?

(a) Run BFS from any uncoloured vertex, colouring layer 0 red, layer 1 blue, layer 2 red, and so on. Repeat until every vertex is coloured. Then look at every edge: if both ends have the same colour, answer no; otherwise yes.

(b) In a two-colouring, colours must alternate along every path, and BFS layers are exactly "distance mod 2", so the layering is the only colouring worth trying (up to swapping colours per component).

(c) If the answer is yes, every edge joins different colours, so the colouring is a witness. If it is no, some edge joins two vertices in layers of the same parity; with the two BFS paths back to their common ancestor, that edge closes an odd cycle, and an odd cycle cannot be two-coloured. So no two-colouring exists.

(d) BFS is O(m + n) over all components; the edge check is O(m). Total O(m + n).

> **Note what (c) did.** Both answers got a proof: "yes" by showing the witness, "no" by showing an obstacle. An algorithm that answers yes or no needs both directions.

The same parts in CLRS. CLRS chapter 2 goes in the same order, without a labelled intuition part: insertion sort is given as pseudocode, proved correct (§2.1), and its running time counted line by line, cost times number of runs (§2.2). Its correctness proofs for loops use a **loop invariant** in three steps (§2.1, pages 18 to 20):

- **Initialization**: it is true before the first iteration;
- **Maintenance**: if it is true before an iteration, it is true before the next;
- **Termination**: when the loop stops, the invariant, with the reason it stopped, shows the output is right.

The first two are an induction. The third is the step that gets forgotten. The course asks for part (b) as well, so write it even when the book you are reading does not.

WARM-UP W1 · GRADE THIS ANSWER

"Algorithm: run Gale-Shapley. It works because the lecture proved it does. It makes at most n^2 proposals." A classmate wrote this for "find a stable matching when n students and n colleges have strict rankings". Which parts are present, and which are missing or too thin?

PART 1 · QUESTIONS

Writing an algorithm answer

1.1 *Skill: telling the four parts apart*

> Every edge is looked at twice, once from each end, and every vertex is put in the queue at most once.

A write-up for "detect a cycle in an undirected graph with BFS" contains this sentence. Which of the four parts is it?

- ◯ **A.** (a) the algorithm
- ◯ **B.** (b) the intuition
- ◯ **C.** (c) correctness
- ◯ **D.** (d) the running time

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.2 *Skill: what counts as a correctness argument*

Which of these, on their own, justify that an algorithm is correct?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Running it on the example from the problem and getting the right answer.
- ☐ **B.** Running it on 1,000 random inputs and getting the right answer each time.
- ☐ **C.** An invariant that holds at the start, is kept by every step, and at the end implies the output is right.
- ☐ **D.** A proof by contradiction: suppose the output is wrong, and derive something impossible.
- ☐ **E.** "It clearly works, since each step is reasonable."

1.3 *Skill: the three steps of a loop invariant (CLRS §2.1)*

```
for i = 1 to n
  if A[i] = v
    return i
return NIL
```

Linear search, as in CLRS Exercise 2.1-3, with the invariant "at the start of the iteration for i, v is not in A[1..i − 1]". CLRS proves an invariant in three steps. In which step does the reason the loop stopped get used?

- ◯ **A.** Initialization: it holds before the first iteration.
- ◯ **B.** Maintenance: if it holds before an iteration, it holds before the next.
- ◯ **C.** Termination: when the loop stops, the invariant gives the answer.
- ◯ **D.** None of them: an invariant that holds before every iteration already proves the output right.

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.4 *Skill: a complete four-part answer*

Give a four-part answer for: given an array of n integers, decide whether any value appears twice. Your algorithm must beat $O(n^2)$.

1.5 *Skill: running time for a problem the question sets*

A problem asks for "an algorithm with running time strictly slower-growing than $f(n) = n^3$, meaning $\lim g(n)/f(n) = 0$". Which g does NOT qualify?

- ◯ **A.** n^2
- ◯ **B.** $n^2 \log n$
- ◯ **C.** n^2.99
- ◯ **D.** $n^3 / 1000$

Why not the others? Pick the one you almost chose and say what is wrong with it.

PART 2 · LEARN

Always, or not always?

"Does there always exist…?" has two possible answers, and each has its own kind of proof. Decide which you believe *before* writing, by trying small instances and trying to break your belief.

Yes, always. Give an algorithm (or reduction) that works on every instance, prove its output has the property, and bound its running time.

No, not always. Give one concrete instance, and prove that *every* candidate for it fails. Not one algorithm's output: every candidate.

Worked "no": stable roommates

Four people share two double rooms. Each ranks the other three. A pairing is unstable if two people in different rooms both prefer each other to their roommates. Does a stable pairing always exist?

Ana	`Bo > Cy > Di`
Bo	`Cy > Ana > Di`
Cy	`Ana > Bo > Di`
Di	`Ana > Bo > Cy`

There are exactly three pairings. Check each:

- **Ana–Bo, Cy–Di**: Bo and Cy both prefer each other to their roommates.
- **Ana–Cy, Bo–Di**: Ana and Bo both prefer each other to their roommates.
- **Ana–Di, Bo–Cy**: Ana and Cy both prefer each other to their roommates.

Every pairing is unstable, so the answer is **no**. The key to finding it: Di is everyone's last choice, so whoever rooms with Di wants out, and the other three prefer each other in a cycle.

Worked "yes": a reduction

If a variant looks like the original problem with a twist, try to **transform** the variant into the original, run the algorithm you already trust, and **translate back**. The proof then says: if the translated answer had an instability, the answer to the original would have had one too, and it does not. Question 2.7 walks you through one.

> **Strategy.** Start with n = 2 (2 matchings) and n = 3 (6 matchings). Try the extremes: rankings that all agree, rankings that all differ, many ties, no ties. If every attempt to break the property fails, look for a proof; if one succeeds, you have your counterexample.

PART 2 · LEARN

Preferences with ties

Rankings may now contain ties: a college can be indifferent between two students. Write a tie in brackets: (A = B) > C means A and B are tied at the top, both above C.

Prefers means strictly: s prefers c_1 to c_2 when c_1 is ranked higher and they are not tied.
For s matched to c′ and c matched to s′, the pair (s, c) is a
- **strong instability** if s prefers c to c′ *and* c prefers s to s′ (both strictly gain);
- **weak instability** if one side prefers, and the other either prefers or is indifferent (one gains, nobody loses).

Every strong instability is also a weak one. So "no weak instability" is the stricter demand.

Worked instance

Student	Ranking, best first
Ava	`(MIT = NYU) > UMass`
Ben	`MIT > NYU > UMass`
Cal	`NYU > (MIT = UMass)`

College	Ranking, best first
MIT	`Cal > (Ava = Ben)`
NYU	`Ava > (Ben = Cal)`
UMass	`Ben > Ava > Cal`

Take the matching **Ava–UMass, Ben–NYU, Cal–MIT**. Check every pair not in it, one side at a time:

- **Ava & NYU**: Ava gains (prefers NYU to UMass); NYU gains (prefers Ava to Ben). **Strong** (and so weak too): both strictly gain.
- **Cal & NYU**: Cal gains (prefers NYU to MIT); NYU is level (indifferent between Cal and Ben). **Weak only**: one gains and the other does not lose.

Every other pair has someone who would lose by the swap, so it is neither. This matching has a strong instability, so it fails both standards.

WARM-UP W2 · ONE SIDE AT A TIME

In the same instance, with the same matching, check the pair Ben & MIT. Does Ben gain, stay level, or lose? Does MIT? So which kind is it, if any?

PART 2 · QUESTIONS

Always, or not always?

2.1 *Skill: what a proof of "not always" needs*

You believe that for some problem, a matching with property P does not always exist. Which of these are needed for a complete answer?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** One specific instance (actual people, actual rankings).
- ☐ **B.** A proof that every matching of that instance fails P.
- ☐ **C.** A proof that the algorithm from lecture fails P on that instance.
- ☐ **D.** Many instances, to be safe.

2.2 *Skill: what a proof of "always" needs*

You believe a matching with property P always exists. Which of these would be complete answers?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** An algorithm that works on every instance, with a proof that its output always has P, and its running time.
- ☐ **B.** A reduction: transform any instance into one a known algorithm solves, run it, translate the answer back, and prove the translated answer has P.
- ☐ **C.** Checking that every instance with 3 students has one.
- ☐ **D.** An instance where one exists.

2.3 *Skill: classifying pairs with ties*

For the instance with ties on the Learn page, take the matching Ava–MIT, Ben–NYU, Cal–UMass. For each of the six pairs not in the matching, say whether it is a strong instability, a weak instability only, or neither, and why.

Pair	Strong	Weak only	Neither	Why (one line)
Ava & NYU				
Ava & UMass				
Ben & MIT				
Ben & UMass				
Cal & MIT				
Cal & NYU				

2.4 *Skill: exhausting every matching*

Still in the instance with ties: how many of its 6 perfect matchings have no weak instability at all?

matchings ☐

Working:

2.5 *Skill: what one instance tells you*

This instance has a matching with no weak instability. What does that tell you about whether every instance with ties has one?

- ◯ **A.** Every instance has one.
- ◯ **B.** Some instance does not.
- ◯ **C.** Nothing either way.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.6 *Skill: the definitions, exactly*

With ties allowed, which of these are true for every instance and every perfect matching?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Every strong instability is also a weak instability.
- ☐ **B.** Every weak instability is also a strong instability.
- ☐ **C.** If nobody has a tie anywhere, strong and weak instabilities are the same pairs.
- ☐ **D.** A matching with no weak instability has no strong instability.

2.7 *Skill: a reduction proof, one step at a time*

Colleges now have k seats each (n colleges, kn students, strict rankings). A pair (s, c) is unstable if s prefers c to their college, and c prefers s to one of the students it holds. Show that a stable assignment always exists by reducing to ordinary stable matching.

Counting when the inner work is not constant

Lecture counted loops whose innermost line is one step. Real programs hide loops inside lines: "scan", "copy", "max of", "is x in the list". The first move is to **rewrite every hidden loop as a loop**, then count as usual.

```
for i = 1 to n
  for j = 1 to i
    total = A[1] + A[2] + … + A[j]      ← a loop of j steps
```

Exact

Pair (i, j) costs j. Summing over $j \le i$ gives $i(i + 1)/2$ (CLRS equation (A.1)), and summing that over i, with (A.1) and (A.3), gives $n(n + 1)(n + 2)/6$. At n = 8: 120 steps (the program, run, agrees for every n up to 40).

Ceiling: add work that is not there

Every loop runs at most n times, so at most $n \cdot n \cdot n = n^3$ steps. $O(n^3)$.

Floor: throw work away

Keep only the pairs with $i \ge n/2$ and $n/4 \le j \le n/2$. Each really runs ($j \le n/2 \le i$), each costs $j \ge n/4$ steps, and there are about $(n/2)(n/4)$ of them. So at least about $n^3/32$ steps: $\Omega(n^3)$. At n = 8: 15 pairs in the box, 45 steps.

Ceiling and floor together: **$\Theta(n^3)$**. The floor did not need the exact count, only a box where the number of i, the number of j and the inner work each grow like n.

> **In CLRS.** Appendix A.1 lists the exact sums by number: (A.1) for $1 + 2 + \dots + n$ and (A.3) for $1^2 + 2^2 + \dots + n^2$ (pages 1146 to 1147). Appendix A.2 has the two shortcuts used here, on one index: **bounding the terms**, every term by the largest (the ceiling, page 1151), and **splitting the summation**, keeping only the part where every term is large (the floor, page 1152). The box is a split with three indices. CLRS §2.2 counts insertion sort the same way: each line's cost times the number of times it runs.

Same answers, less work

Suppose you need $P[k] = x^k$ for every k from 1 to n. Computing each from scratch takes $k - 1$ multiplications: 28 at n = 8, $\Theta(n^2)$ in all. But x^k is x^{k-1} times x, and x^{k-1} is already in the table: **$P[k] = P[k - 1] \cdot x$**, one multiplication each, 7 at n = 8, $\Theta(n)$.

> **Where the speed-up came from.** The slow version redid work it had just done. Ask of any slow loop: can this answer be built from answers I already have? Then keep them.

> **The output is a floor.** An algorithm that must write n answers is $\Omega(n)$, so the reuse version of the powers table is as fast as that job can go. Writing a whole $n \times n$ table is $\Omega(n^2)$, however clever the algorithm.

WARM-UP W3 · REWRITE, THEN COUNT

```
for i = 1 to n
  for j = 1 to n
    if A[j] is in A[1..i] then count++
```

Rewrite the "is in" as a loop, and give a ceiling and a floor. (Assume the search scans A[1..i] in full.)

PART 3 · QUESTIONS

Counting when the inner work is not constant

3.1 *Skill: an exact count with a dependent inner loop*

```
for i = 1 to n
  for j = 1 to n
    for k = 1 to min(i, j)
      one step
```

How many times does the innermost line run, at n = 6?

steps ☐

Working:

3.2 *Skill: counting triples*

```
for i = 1 to n
  for j = i + 1 to n
    for k = j + 1 to n
      one step
```

How many times does "one step" run at n = 6?

steps ☐

Working:

3.3 *Skill: a floor on a sum by splitting it (CLRS §A.2)*

Find a floor for $1^2 + 2^2 + \dots + n^2$ the way CLRS Appendix A.2 bounds a sum by splitting it: throw away the terms with $k \le n/2$, and bound each term that is left from below by $(n/2)^2$. What floor does that give at n = 12?

floor ☐

Working:

3.4 *Skill: a floor by throwing work away*

For the program on the Learn page (for i, for j = 1..i, a loop of j steps), keep only the pairs with $i \geq n/2$ and $n/4 \leq j \leq n/2$. (a) Why do all these pairs really run? (b) How many steps does each of their inner loops take, at least? (c) How many such pairs are there, at least? (d) Conclude a floor, and fill in the numbers at n = 12.

3.5 *Skill: why the box has to be in the middle*

Someone tries the floor for the same program with the box "$i \geq n/2$ and $j \leq 10$". What goes wrong?

- ◯ **A.** Nothing: it gives $\Omega(n^3)$.
- ◯ **B.** It only gives $\Omega(n)$, because the j-range and the inner work stay constant as n grows.
- ◯ **C.** Those pairs do not run.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.6 *Skill: doing the same job with less work*

Fill in the table C[i, j] = "i choose j" for every $0 \leq j \leq i \leq n$. The obvious way computes each entry from its factorial formula, $i! / (j!\,(i-j)!)$, building each factorial by multiplying from 1. Give a four-part answer for a faster algorithm, and say how much faster.

3.7 *Skill: the size of the output is a floor*

An algorithm must fill in an n × n table of answers (one entry for every pair). What is the best running time it could possibly have?

- ◯ **A.** O(n), with a clever enough idea.
- ◯ **B.** O(n log n)
- ◯ **C.** $\Omega(n^2)$: it cannot beat the cost of writing its output.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.8 *Skill: not every line is one step*

Which lines take more than constant time, so that counting them as one step would undercount?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** sum = sum + A[i]
- ☐ **B.** m = the largest value in list L
- ☐ **C.** if x is in list L
- ☐ **D.** B[i, j] = 0
- ☐ **E.** copy A[1..i] into C

PART 4 · LEARN

Counting proposals

Lecture showed propose-and-reject makes at most n^2 proposals: each student proposes to each college at most once. That is a ceiling. How many it *actually* makes depends on the rankings.

Worked instance

Student	1st	2nd	3rd
Ava	MIT	NYU	UMass
Ben	MIT	UMass	NYU
Cal	NYU	MIT	UMass

College	1st	2nd	3rd
MIT	Ben	Ava	Cal
NYU	Ava	Cal	Ben
UMass	Cal	Ben	Ava

#	asks	whom	what happens
1	Ava	MIT	MIT was holding nobody, so holds Ava.
2	Ben	MIT	MIT likes Ben better than Ava: holds Ben, **lets Ava go**.
3	Ava	NYU	NYU was holding nobody, so holds Ava.
4	Cal	NYU	Turned away: NYU is holding Ava and likes them better.
5	Cal	MIT	Turned away: MIT is holding Ben and likes them better.
6	Cal	UMass	UMass was holding nobody, so holds Cal.

6 proposals, against a ceiling of 9. Look at where each student ended up on their own list, and add the positions: you get the same number. Question 4.3 asks you to explain why that always works.

> **Two kinds of claim.** "At most n^2" is a ceiling for every instance. "It can take $\Omega(n^2)$" or "it can finish in $O(n)$" are claims about the *best* or *worst* instance, and are proved by giving, for every n, an instance and its count. CLRS §3.1 (page 49) makes the same distinction: an Ω bound on the running time, with no modifier, holds for every input, while an Ω bound on the worst case needs only one bad input for each n.

WARM-UP W4 · CEILING OR NOT

Could any instance with n = 3 need 10 proposals? Why or why not?

PART 4 · QUESTIONS

Counting proposals

4.1 *Skill: counting proposals by hand*

Student	1st	2nd	3rd
Ava	NYU	MIT	UMass
Ben	NYU	UMass	MIT
Cal	MIT	NYU	UMass

College	1st	2nd	3rd
MIT	Cal	Ava	Ben
NYU	Ava	Cal	Ben
UMass	Ben	Ava	Cal

Students propose. How many proposals happen in total, including the ones that are turned down?

proposals ☐

Working:

4.2 *Skill: the same count, a bigger instance*

Student	1st	2nd	3rd	4th
Ava	MIT	NYU	UMass	Yale
Ben	NYU	MIT	Yale	UMass
Cal	MIT	UMass	NYU	Yale
Dee	UMass	Yale	MIT	NYU

College	1st	2nd	3rd	4th
MIT	Dee	Ben	Ava	Cal
NYU	Ava	Cal	Dee	Ben
UMass	Ben	Dee	Cal	Ava
Yale	Cal	Ava	Ben	Dee

Students propose. How many proposals in total?

proposals ☐

Working:

4.3 *Skill: turning a run into a count*

Explain why, when students propose, the total number of proposals equals the sum over students of the position of their final college on their own list (1 for first choice). Then check it on question 4.1.

4.4 *Skill: building an instance to order*

With n = 3, write student and college lists for which the students-propose run makes exactly 4 proposals, and a second pair of lists that makes exactly 6. Check each by tracing.

4 proposals

Student	1st	2nd	3rd
Ava			
Ben			
Cal			

College	1st	2nd	3rd
MIT			
NYU			
UMass			

6 proposals

Student	1st	2nd	3rd
Ava			
Ben			
Cal			

College	1st	2nd	3rd
MIT			
NYU			
UMass			

4.5 *Skill: what a bound on the count needs*

To show the algorithm can take $\Omega(n^2)$ proposals, what do you have to give?

- ◯ **A.** One instance with n = 100 that takes 5,000 proposals.
- ◯ **B.** For every n, an instance of that size, and a proof that it takes at least $c \cdot n^2$ proposals for some constant $c > 0$.
- ◯ **C.** A proof that every instance takes at least $c \cdot n^2$ proposals.

Why not the others? Pick the one you almost chose and say what is wrong with it.

PART 5 · LEARN

Searching when failures are limited

A board holds every weight up to its strength and breaks at anything heavier. Weights are 1..n; the strength is some number from 0 to n. A broken board is gone. You want the strength in as few tests as possible, *in the worst case*, and you must always get it right.

Boards	Strategy	Worst case
1	1, 2, 3, … until it breaks. Forced: skip a weight and a break leaves you unable to tell.	n
Unlimited	Binary search: test the middle of what is still possible.	about $\log_2 n$
2	?	?

What a two-board plan looks like

Every two-board plan has the same shape: the first board tests some increasing weights $w_1 < w_2 < \ldots$; when it breaks at w_k, the second (last) board must scan from $w_{k-1} + 1$ upward one at a time. So a plan is just the list of first-board weights, and its cost for a given strength is

```
tests = (first-board tests up to the break) + (second-board scan in that gap)
```

Worst case, adversary style

Think of the strength as chosen by an adversary who has seen your plan. For each gap, they can make the first board break at its top and the second board scan the whole gap. The worst case is the costliest gap: its position in the list plus its width.

> **The tension.** Wide gaps make the second board scan a lot. Narrow gaps mean many first-board tests before you reach the later gaps. A good plan balances the two. How exactly is for you to find: questions 5.5 to 5.7 give you the numbers to reason from.

WARM-UP W5 · TWO SMALL PLANS

n = 10. Compare the plans "5" and "3, 6, 9": for each, write the cost of every gap and the worst case.

PART 5 · QUESTIONS

Searching when failures are limited

5.1 *Skill: why one board forces a scan*

With one board and weights 1..n, why can you not test 1, 3, 5, … to save time?

- ◯ **A.** Odd weights are harder to set up.
- ◯ **B.** If it breaks at 3, the strength could be 1 or 2, and you have no board left to tell which.
- ◯ **C.** You can; it halves the tests.

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.2 *Skill: binary search, when boards are free*

With as many boards as you like, n = 100, how many tests does binary search need in the worst case? (There are 101 possible answers, 0 to 100.)

tests ☐

Working:

5.3 *Skill: binary search with only two boards*

With exactly two boards and n = 100, you start binary search at 50. Which of these are true?

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** If the first board holds at 50, you still have two boards and the range 51..100.
- ☐ **B.** If it breaks at 50, you have one board for the range 1..49.
- ☐ **C.** If it breaks at 50, the worst case is still about $\log_2 100$ tests.
- ☐ **D.** Starting in the middle is the best first test.

5.4 *Skill: playing a plan against one strength*

n = 30. First board tests 12, 20, 26, then the second board scans up from the last weight that held. If the true strength is 17, how many tests happen?

tests ☐

Working:

5.5 *Skill: the worst case of a plan*

Same plan (n = 30, first board: 12, 20, 26). What is the worst-case number of tests over every strength from 0 to 30?

tests ☐

Working:

5.6 *Skill: improving a plan by one move*

n = 100, first board: 20, 50, 70, 85, worst case 31 tests (at strength 48 and 49). Moving which one listed weight (keeping the list in order) lowers the worst case the most?

- ◯ **A.** Move 20
- ◯ **B.** Move 50
- ◯ **C.** Move 70
- ◯ **D.** Move 85

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.7 *Skill: a plan with few first-board tests*

n = 100, first board: 40, 80. Worst case?

tests ☐

Working:

KEEP THIS PAGE

One-page summary

Writing it up

- Four labelled parts: algorithm, intuition, correctness, running time.
- Correctness covers every input. Examples are not proofs.
- Yes/no answers: prove both directions.
- Loop invariant (CLRS §2.1): initialization, maintenance, termination. Termination uses the reason the loop stopped.
- "Strictly slower" means $g/f \to 0$: a different growth class, not a smaller constant.

Always or not

- Try n = 2 and 3 first; try to break your guess.
- No: one concrete instance, every candidate ruled out.
- Yes: algorithm or reduction, with proof and time.
- Reduction: transform, run what you trust, translate back, prove an instability would carry over.

Ties

- Prefers = strictly. Indifferent = tied.
- Strong: both gain. Weak: one gains, the other does not lose.
- Strong $\subseteq$ weak. No weak ones is the harder demand.
- Check one side at a time: gain, level, or lose.

Counting

- Rewrite hidden loops (scan, copy, max, search) as loops.
- Ceiling: every loop runs to n (CLRS A.2: bound each term by the largest).
- Floor: keep a box where the number of each index and the inner work all grow like n (CLRS A.2: split the sum).
- Exact sums: CLRS (A.1) and (A.3).
- Output size is a floor on any algorithm.
- Faster: build each answer from answers you already have.

Proposals

- Count every proposal, turned down or not.
- Total = sum of each student's final position on their own list.
- Best/worst-case claims need an instance for every n, described by a rule, with its count proved.

Limited failures

- Last board: scan one at a time.
- Plenty of boards: binary search, and no plan beats it, since t tests have at most 2^t outcomes (CLRS §8.1).
- Plan = first-board weights; cost = tests so far + gap width.
- Worst case = costliest gap. Balance the gaps against the tests.

BEFORE A CHALLENGE SET

Ready-check

Tick each one you could do cold, without this packet. If one is not ticked, the part it comes from is where to go back to.

- [ ] I can write a four-part answer where each part would survive a classmate checking it.
- [ ] Given a variant of stable matching, I try n = 2 and n = 3 instances before deciding "always" or "not always".
- [ ] I can list every perfect matching of a 3 × 3 instance and classify each pair as strong, weak only, or neither, quickly.
- [ ] I can write a reduction to ordinary stable matching and prove that an instability would carry over.
- [ ] I can rewrite a line that hides a loop, and give an exact count, a ceiling and a box-style floor for three nested loops.
- [ ] I can find a faster algorithm by building answers from ones already computed, and explain why it cannot beat the size of its output.
- [ ] I can count proposals from where each student ends up, and I can build an instance that makes a chosen number of them.
- [ ] I can say what a best-case or worst-case bound on proposals needs: an instance for every n, and a proof of its count.
- [ ] I can compute the worst case of a two-board plan and say which change to it helps.

ANSWER KEY

Answer key: warm-ups

Try everything before you look.

W1 · grading an answer

(a) is there only by name: "run Gale-Shapley" is fine if the course has defined it, but say who proposes. (b) is missing. (c) is a citation, not an argument; if the question is about the lecture algorithm itself that is circular, and for any variant it is not enough. (d) says "at most n^2 proposals" but not what each costs: O(1) per proposal with a rank table, so $O(n^2)$ in all.

W2 · Ben & MIT

Ben & MIT: Ben gains (prefers MIT to NYU); MIT loses (prefers Cal to Ben). **Neither**: someone would lose by the swap, or nobody gains.

W3 · rewrite, then count

The test is a loop of i steps, so pair (i, j) costs i, and there are n pairs for each i: $n \cdot (1 + 2 + \dots + n) = n^2(n + 1)/2$ steps. Ceiling n^3. Floor: keep $i \geq n/2$, any j: $n/2 \cdot n$ pairs, each $\geq n/2$ steps, so $\geq n^3/4$. $\Theta(n^3)$. (At n = 8: 288 steps.)

W4 · 10 proposals at n = 3?

No. Each student proposes to each college at most once, so at most $3 \cdot 3 = 9$. (In fact at most 7: the run stops the moment the last college gets its first proposal, so that college gets exactly one, and the other two get at most three each.)

W5 · two plans at n = 10

Plan "5": gap costs 1..4: 5; 6..10: 6. Worst case 6.

Plan "3, 6, 9": gap costs 1..2: 3; 4..5: 4; 7..8: 5; 10: 4. Worst case 5.

Each gap's cost includes the first-board test that broke at its top. The scan inside a gap stops one short of the top, since the top already broke.

ANSWER KEY

Answer key: questions

1.1

Answer: **D**

- **A** × It describes what the algorithm does, but only in order to count it. The algorithm is the steps, not their cost.
- **B** × Intuition says why the idea should work, not how long it takes.
- **C** × Nothing here says the output is right: a wrong algorithm could touch every edge twice too.
- **D** ✓ It bounds the work: O(1) per queue visit and per edge end, so O(m + n).

Takeaway. Before handing in, label each paragraph (a), (b), (c) or (d) in the margin. A part with no paragraph is a part you have not written.

1.2

True: **C, D** · False: A, B, E

- **A** × One instance shows it works on one instance. Correctness is a claim about every input.
- **B** × Evidence, not proof. Bugs live in the inputs you did not generate: the empty one, the tied one, the one where everything is equal.
- **C** ✓ This is a proof, and it covers every input at once. CLRS §2.1 calls the three steps initialization, maintenance and termination.
- **D** ✓ Also a proof. Gale-Shapley's stability proof is this shape: suppose a blocking pair, then someone proposed where they could not have.
- **E** × This is intuition, part (b). Useful, but the grader asked for (c) as well.

Takeaway. Examples find bugs. Only an argument about every input shows there are none.

1.3

Answer: **C**

- **A** × At i = 1 the range A[1..0] is empty, so the invariant holds trivially. Nothing has stopped yet.
- **B** × This step uses what one iteration does: the loop only goes on to i + 1 when A[i] ≠ v, so v is not in A[1..i] either.
- **C** ✓ There are two ways out. Returning i happens only when A[i] = v, so i is right. Leaving the loop means i = n + 1, and the invariant then says v is not in A[1..n], so NIL is right.
- **D** × Holding at every step says nothing about the output until you look at how the loop ends. CLRS calls termination "perhaps the most important" of the three.

Takeaway. Initialization and maintenance are the base case and the step of an induction. Termination is the extra step that turns the invariant into the answer, and it is the one most often left out.

1.4

(a) Sort the array. Then compare each entry with the next one; answer yes if some A[i] = A[i + 1], and no otherwise.

(b) Sorting puts equal values next to each other, so a duplicate, if there is one, must show up as two equal neighbours.

(c) If the answer is yes, the two equal neighbours are a duplicate, so yes is right. If some value v appears twice, then after sorting all the copies of v are consecutive (anything between two copies would be ≥ v and ≤ v, so equal to v), so some neighbouring pair is equal and the algorithm says yes.

(d) Sorting is O(n log n) (merge sort), and the scan is n − 1 comparisons, so O(n log n) in total, which is $o(n^2)$.

Takeaway. Notice that (c) has two directions: every yes is right, and every true duplicate is found. Checking only one is the most common gap.

1.5

Answer: **D**

- **A** × $n^2/n^3 = 1/n \to 0$. Qualifies.
- **B** × $(n^2 \log n)/n^3 = (\log n)/n \to 0$. Qualifies.
- **C** × n^2.99/n^3 = n^(−0.01) → 0, slowly, but it does. Qualifies.
- **D** ✓ The ratio is 1/1000, a constant, not 0. Dividing by a constant never changes the growth: this is still $\Theta(n^3)$.

Takeaway. Strictly slower means a different growth class, not a smaller constant. CLRS §3.1 writes this g(n) = $o(n^3)$. It defines o with "for any positive constant c > 0" (page 50), and gives the limit as equation (3.1) (page 51), noting that some authors use the limit as the definition. For these g the two say the

same.

2.1

True: **A, B** · False: C, D

- **A** ✓ "Not always" is disproved by a single instance, so you must give it, concretely.
- **B** ✓ Showing that one matching fails proves nothing: a different matching of the same instance might have P.
- **C** × One algorithm failing does not mean no matching has P. You must rule out every matching, not one algorithm's output.
- **D** × One is enough, if the proof covers all of its matchings. More instances add length, not strength.

Takeaway. Small is your friend: with 2 students there are 2 perfect matchings, with 3 there are 6. Check them all, one line each.

2.2

True: **A, B** · False: C, D

- **A** ✓ This is the constructive route, and when the question says "give and analyze an algorithm" it is the route they want.
- **B** ✓ A reduction is an algorithm too. The proof burden is the translation step: an instability in your answer must give an instability the known algorithm could not have produced.
- **C** × Every size, not one size. Small cases are how you form the belief, not how you prove it.
- **D** × Existence in one instance says nothing about the others.

Takeaway. Before proving, decide which one you believe by trying small instances. Try to break it first: a counterexample is shorter than a proof.

2.3

- **Ava & NYU**: Ava is level (indifferent between NYU and MIT); NYU gains (prefers Ava to Ben). **Weak only**: one gains and the other does not lose.
- **Ava & UMass**: Ava loses (prefers MIT to UMass); UMass gains (prefers Ava to Cal). **Neither**: someone would lose by the swap, or nobody gains.
- **Ben & MIT**: Ben gains (prefers MIT to NYU); MIT is level (indifferent between Ben and Ava). **Weak only**: one gains and the other does not lose.
- **Ben & UMass**: Ben loses (prefers NYU to UMass); UMass gains (prefers Ben to Cal). **Neither**: someone would lose by the swap, or nobody gains.
- **Cal & MIT**: Cal is level (indifferent between MIT and UMass); MIT gains (prefers Cal to Ava). **Weak only**: one gains and the other does not lose.
- **Cal & NYU**: Cal gains (prefers NYU to UMass); NYU is level (indifferent between Cal and Ben). **Weak only**: one gains and the other does not lose.

So this matching has 4 instabilities, all weak only: it has **no strong** instability, but it is not free of weak ones.

Takeaway. Work one side at a time: first "does the student strictly gain, stay level, or lose?", then the same for the college. The answer is then a lookup.

2.4

Answer: **matchings 1**

List all six and classify each (one line each is enough). Ava–UMass, Ben–NYU, Cal–MIT: Ava & NYU (strong), Cal & NYU (weak). Ava–NYU, Ben–UMass, Cal–MIT: none. Ava–NYU, Ben–MIT, Cal–UMass: Cal & MIT (weak). Ava–UMass, Ben–MIT, Cal–NYU: Ava & MIT (weak), Ava & NYU (strong). Ava–MIT, Ben–UMass, Cal–NYU: Ava & NYU (weak), Ben & MIT (weak), Ben & NYU (weak). Ava–MIT, Ben–NYU, Cal–UMass: Ava & NYU (weak), Ben & MIT (weak), Cal & MIT (weak), Cal & NYU (weak). Only Ava–NYU, Ben–UMass, Cal–MIT is clean.

- **If you wrote 4:** That is how many have no strong instability. A matching can be free of strong ones and still have weak ones.
- **If you wrote 0:** Check the matching the key names: every pair outside it has someone who would lose.

Takeaway. Exhausting every matching is exactly what a counterexample proof has to do, so it is worth being fast at it.

2.5

Answer: **C**

- **A** × One instance cannot prove "every". That needs an argument about all of them.
- **B** × Nothing here points that way either.
- **C** ✓ An instance that has one is consistent with both answers. Only a proof, or a counterexample, settles it.

Takeaway. When exploring, keep a note of which instances you tried and why. Extreme instances are where surprises live.

2.6

True: **A, C, D** · False: B

- **A** ✓ Strong: both strictly prefer. Then in particular one strictly prefers and the other strictly prefers, which the weak definition allows.
- **B** × No. In the Learn page's matching, Cal gains (prefers NYU to MIT) and NYU is level (indifferent between Cal and Ben): weak, not strong.
- **C** ✓ With no ties, "indifferent" never happens, so "prefers or indifferent" is just "prefers".
- **D** ✓ Strong ones are weak ones, so having none of the weak kind means none of the strong kind.

Takeaway. Weak instabilities are the larger set. Avoiding all of them is the harder demand.

2.7

Transform. Replace each college c by k copies $c^1 \dots c^k$, each with c's ranking of students. Each student ranks the copies in place of c, in the order $c^1 > c^2 > \dots > c^k$, keeping the other colleges where they were. Now there are kn students and kn seats: an ordinary instance.

Run Gale-Shapley; it returns a stable matching M′. **Translate back:** s gets c if s is matched to any copy of c. Every college gets exactly k students.

Prove. Suppose (s, c) were unstable in the translation: s prefers c to its college d, and c prefers s to some s′ it holds, say s′ is on copy c^i. In the copied instance, s ranks c^i above its own copy d^j (every copy of c is above every copy of d), and c^i ranks s above s′. So (s, c^i) blocks M′, contradicting its stability.

Time. The copied instance has kn on each side, so $O((kn)^2)$.

Takeaway. The shape to remember: transform, run what you know, translate back, and prove "an instability in the translation would be an instability the known algorithm never leaves".

3.1

Answer: **steps 91**

For each (i, j) the inner loop runs min(i, j) times. Group by the value m = min(i, j): it equals m on $2(n - m) + 1$ pairs. Or count by k: step k runs for every (i, j) with both $\geq k$, which is $(n - k + 1)^2$ pairs, so the total is $1^2 + 2^2 + \dots + n^2 = n(n + 1)(2n + 1)/6 = 91$ at n = 6. That sum is CLRS equation (A.3).

- **If you wrote 216:** That is n^3: it assumes every inner loop runs n times. That is the upper bound, not the count.
- **If you wrote 126:** That treats the inner loop as running i times (or j). It runs min(i, j) times, which is smaller.

Takeaway. Swapping the order of summation (count by k instead of by (i, j)) is often the shortcut to a closed form.

3.2

Answer: **steps 20**

Each step is one triple $i < j < k$ chosen from 1..n, and each triple is visited once: $C(n, 3) = n(n - 1)(n - 2)/6 = 20$, the number of ways to choose 3 from n, CLRS equation (C.2) with k = 3.

- **If you wrote 216:** n^3 counts ordered triples with repeats, like (2, 2, 5) and (5, 2, 2).
- **If you wrote 120:** That counts ordered triples of distinct values. Each set {i, j, k} is visited once, in increasing order, not 3! = 6 times.

Takeaway. $\Theta(n^3)$, with constant 1/6. A floor argument does not need the constant, only that it is not zero.

3.3

Answer: **floor 216**

The terms left are k = 7 to 12: 6 of them, each at least $6^2 = 36$. So the sum is at least $6 \cdot 36 = 216$, which is $n^3/8$. The exact sum is $n(n + 1)(2n + 1)/6 = 650$, CLRS equation (A.3).

- **If you wrote 12:** That bounds every term by the smallest one, 1^2, and keeps them all: n. CLRS shows this failing on its own example: the smallest term is too small to be useful. Splitting first is the fix.
- **If you wrote 650:** That is the exact sum, (A.3). The question asks for the floor the split gives, which is smaller and much easier to get.
- **If you wrote 294:** That uses 7^2, the smallest term kept. It is a correct floor too, but the question fixed $(n/2)^2$, so that the floor comes out as $n^3/8$ for every even n.

Takeaway. The ceiling: n terms, each at most n^2, so n^3 (CLRS calls this bounding the terms). With the floor $n^3/8$, the sum is $\Theta(n^3)$. The box in the next question is the same split, with three indices.

3.4

(a) $j \le n/2 \le i$, so j is in the range 1..i: the program really visits (i, j). (b) Its inner loop is j steps, and $j \ge n/4$. (c) At least n/2 choices of i and about n/4 choices of j, so about $n^2/8$ pairs. (d) Steps $\ge (n^2/8)(n/4) = n^3/32$, so $\Omega(n^3)$; with the upper bound n^3 (every loop runs to n), $\Theta(n^3)$.

At n = 12: 28 pairs, inner loops of at least 3 steps, so at least 84; the box really holds 126 and the whole program 364. The floor is loose, and that is fine: it only has to be a constant times n^3.

Takeaway. Choose a box where three things are each at least a constant fraction of n: the number of i, the number of j, and the inner work. Then multiply.

3.5

Answer: **B**

- **A** × Count it: about n/2 values of i, 10 values of j, inner loops of at most 10 steps. That is at most 50n steps.
- **B** ✓ A floor argument is only as strong as the part you keep. Every factor has to grow with n.
- **C** × They do run ($j \le 10 \le i$ once $n \ge 20$). They just do not carry enough work.

Takeaway. A lower bound you can prove but that is too low is not wrong, just useless. Check each factor grows.

3.6

(a) For i = 0 to n: set C[i, 0] = C[i, i] = 1, then for j = 1 to i − 1 set C[i, j] = C[i − 1, j − 1] + C[i − 1, j].

(b) Row i can be built from row i − 1, which is already in the table: choosing j from i items either uses item i (and j − 1 from the rest) or does not (and j from the rest). CLRS Exercise C.1-7 asks for exactly this proof, and Exercise C.1-8 builds the table, Pascal's triangle.

(c) Induction on i. Row 0 is right. If row i − 1 is right, each entry of row i is the sum of two right entries, and Pascal's rule (the sentence in (b)) says that sum is i choose j.

(d) One addition per entry, $(n + 1)(n + 2)/2$ entries: $\Theta(n^2)$. The factorial way costs about 2i multiplications per entry of row i, 880 at n = 10 against 66 entries, $\Theta(n^3)$. The ratio $\to 0$. And $\Theta(n^2)$ is the best possible here, since there are $\Theta(n^2)$ entries to write.

Takeaway. Look for an answer you can build from answers already in your table. Then each one costs O(1) instead of a fresh computation.

3.7

Answer: **C**

- **A** × Writing n^2 entries takes n^2 steps, whatever the idea.
- **B** × Still below the n^2 writes it cannot avoid.
- **C** ✓ Every entry is at least one step. So $\Theta(n^2)$ is as good as it gets, and an $O(n^2)$ algorithm for it is optimal.

Takeaway. When a problem's answer is a table of every pair, $O(n^2)$ is not a failure to be clever: it is the floor.

3.8

True: **B, C, E** · False: A, D

- **A** × One addition, one read, one write. Constant.
- **B** ✓ Finding a maximum has to look at all |L| entries: that line is a hidden loop.
- **C** ✓ Searching a list is up to |L| comparisons, unless L is a hash set or you are told otherwise.
- **D** × One write into a table. Constant.
- **E** ✓ Copying i entries takes i steps.

Takeaway. Before counting, rewrite every line that hides a loop as the loop. Then count loops as usual. CLRS makes the same warning in §2.2 (footnote 5, page 25): a step written in English, like "sort the points", can take more than constant time.

4.1

Answer: **proposals 4**

Trace it: Ava → NYU, Ben → NYU, Ben → UMass, Cal → MIT.

- **If you wrote 3:** That is only the first round, one proposal per student. Count the ones after a student is let go, too.
- **If you wrote 9:** $n^2 = 9$ is the upper bound from lecture, not the count here.

Takeaway. Count every proposal, accepted or not: the running time is the number of proposals, up to a constant.

4.2

Answer: **proposals 9**

Trace: Ava → MIT, Ben → NYU, Cal → MIT, Cal → UMass, Dee → UMass, Cal → NYU, Ben → MIT, Ava → NYU, Cal → Yale. Check: the final partners sit at positions 2, 2, 4, 1 on the students' lists, which add to 9.

- **If you wrote 16:** $n^2 = 16$ is the lecture's ceiling. Few instances reach it.

Takeaway. The check in the key is the next question.

4.3

Each student proposes down their list in order and never skips a college or proposes twice. The run ends with every student at their final college, which they proposed to last. So a student whose final college is at position p made exactly p proposals, and the total is the sum of the positions. (The order in which free students are chosen does not change the final matching, from lecture, so it does not change this sum either.)

For 4.1: positions 1 + 2 + 1 = 4, the same as the 4 proposals in the trace.

Takeaway. Now "how many proposals?" becomes "where does each student end up on their own list?", which you can often see without tracing.

4.4

Any lists whose trace makes that many proposals are right. Use 4.3 to aim: choose where each student should end up on their own list so the positions add to the target, then pick college rankings that force it. One instance of each, found by the engine:

4 proposals (trace: Ava → MIT, Ben → UMass, Cal → UMass, Ben → NYU)

Student	1st	2nd	3rd
Ava	MIT	UMass	NYU
Ben	UMass	NYU	MIT
Cal	UMass	NYU	MIT

College	1st	2nd	3rd
MIT	Cal	Ben	Ava
NYU	Cal	Ben	Ava
UMass	Cal	Ben	Ava

6 proposals (trace: Ava → UMass, Ben → UMass, Ava → NYU, Cal → UMass, Ben → NYU, Ava → MIT)

Student	1st	2nd	3rd
Ava	UMass	NYU	MIT
Ben	UMass	NYU	MIT
Cal	UMass	NYU	MIT

College	1st	2nd	3rd
MIT	Cal	Ben	Ava
NYU	Cal	Ben	Ava
UMass	Cal	Ben	Ava

Over all 46,656 instances with n = 3, the fewest proposals is 3 and the most is 7, which is less than $n^2 = 9$.

Takeaway. Building an instance to hit a target is a skill of its own: decide the outcome first, then write the rankings that force it.

4.5

Answer: **B**

- **A** × One n says nothing about growth. Ω is a claim about all large n.
- **B** ✓ A family of instances, one per n, with a count you can prove, not just observe.
- **C** × More than a worst-case bound asks for, and false besides: question 4.1's instance takes only 4 proposals.

Takeaway. A lower bound on the worst case needs one bad instance per n, described by a rule, with its count proved. CLRS §3.1 (page 49) draws the same line for insertion sort: its worst-case running time is $\Omega(n^2)$, but its running time is not, since a sorted input takes only linear time.

5.1

Answer: **B**

- **A** × The weights are identical one-pound units; nothing is harder.

- **B** ✓ Skipping a weight is a gamble you can only take if you can afford to lose the board and still finish.
- **C** × It halves the tests and sometimes gets the answer wrong. A strategy must always be right.

Takeaway. With your last board, you must go up one weight at a time. Every plan ends in this phase.

5.2

Answer: **tests 7**

Each test at least halves the range of possible strengths, in the worst case. $2^6 = 64 < 101 \le 128 = 2^7$, so 7. No plan does better, however many boards it has: 6 tests, each holding or breaking, can come out at most $2^6 = 64$ ways, and 101 strengths need 101 different ones. That is the decision-tree argument of CLRS §8.1 (the proof of Theorem 8.1, page 193), with strengths in place of orderings. CLRS Exercise 2.3-5 asks for binary search's worst case as $\Theta(\lg n)$, where lg is $\log_2$.

- **If you wrote 6:** $2^6 = 64$ is fewer than the 101 possible answers.

Takeaway. $\log_2 n$ with unlimited boards, n with one board. The interesting cases are in between.

5.3

True: **A, B** · False: C, D

- **A** ✓ Holding costs nothing. You are in the same situation with half the range.
- **B** ✓ So from there you must scan 1, 2, 3, … one at a time.
- **C** × No: after a break at 50 the last board scans up to 49 weights. The worst case is around 50.
- **D** × A break is expensive and a hold is cheap, so the two outcomes are not worth the same. The best first test is not where binary search puts it.

Takeaway. Binary search treats the two outcomes as equally costly. With a limited number of boards, they are not.

5.4

Answer: **tests 8**

12 holds, 20 breaks (2 tests). The second board tests 13, 14, …, 18: holds up to 17, breaks at 18 (6 more). Total 8.

- **If you wrote 7:** You stopped at 17. You only know 17 is the strength once 18 breaks.

Takeaway. Simulate before optimising: write out the tests for one strength, then find the strength that makes the count largest.

5.5

Answer: **tests 12**

Cost of each gap (first-board tests to reach its top, plus the second board's scan inside it): 1..11: 12; 13..19: 9; 21..25: 8; 27..30: 7. The worst is 12, at strength 10 and 11.

- **If you wrote 11:** You forgot the first board's own test at 12 that broke.

Takeaway. Each gap's cost is "how many first-board tests got you there, plus how wide the gap is". The worst case is the biggest of those.

5.6

Answer: **A**

- **A** ✓ Best is to 25: worst case 26. The costliest gap was 21..49: 31; this move splits its cost more evenly with its neighbour.
- **B** × Helps, but less: the best place for it is 45, worst case 27.
- **C** × Cannot help: wherever it goes, the worst case stays at 31 or more, because the costliest gap does not move.
- **D** × Cannot help: wherever it goes, the worst case stays at 31 or more, because the costliest gap does not move.

Takeaway. The worst case always lives in one gap. Look there first.

5.7

Answer: **tests 41**

Gap costs: 1..39: 40; 41..79: 41; 81..100: 22. The worst is 41, at strength 78 and 79: the gaps are wide, so the second board does most of the work.

Takeaway. Too few first-board tests leaves wide gaps for the second board. Put 5.5, 5.6 and this one side by side and ask what the best plans have in

common.